

Obeo SmartEA - Guide AQL

v7.6.0



1. Introduction	4
2. Requêtes simples	8
2.1. Premiers pas	8
2.2. Appel de services	9
3. Manipulation des chaînes de caractères	10
4. Opérations booléennes	11
5. Manipulation des collections	12
6. Services génériques	14
6.1. Services sur le type	14
6.2. Services de navigation	14
6.3. Services sur EObject	18
7. Services SmartEA	20
7.1. Retrouver le label d'un objet	20
7.2. Naviguer dans les répertoires.	20
7.3. Retrouver le chemin absolu d'un répertoire	20
7.4. Retrouver le parent d'un répertoire	20
7.5. Retrouver un ensemble d'éléments d'un ou plusieurs types donnés	20
7.6. Retrouver un ensemble d'objets ayant une propriété dynamique donnée	21
7.7. Retrouver un ensemble d'objets ayant une propriété dynamique donnée avec une valeur donnée	21
7.8. Retrouver la valeur d'une propriété dynamique donnée	22
7.9. Retrouver une propriété dynamique donnée.	22
7.10. Retrouver la valeur d'une propriété dynamique	22
7.11. Tester la présence d'une propriété dynamique donnée	22
7.12. Tester la correction d'une définition de propriété donnée vis-à-vis d'une liste de valeur	23
7.13. Tester si une valeur est conforme à la contrainte d'une propriété typée	23
7.14. Tester qu'une contrainte est applicable sur un type de propriété typée	23
7.15. Transformer un type de propriété typée en chaîne de caractères	23
7.16. Vérifier qu'une propriété typée est d'un certain type	24
7.17. Retrouver un objet à partir de son identifiant logique	24
7.18. Retrouver les artefacts associés à un objet	24
7.19. Retrouver les artefacts où un objet apparaît	24
7.20. Tester si un élément est stéréotypé par un stéréotype particulier	25
7.21. Retrouver les éléments sémantiques et les artefacts liés à un artefact donné.	25

7.22. Récupérer l'identifiant du projet courant	25
7.23. Récupérer l'identifiant de la branche courante	26
7.24. Récupérer le prisme courant	26
7.25. Récupérer l'url d'un élément sémantique	26
7.26. Récupérer les informations de traçabilité d'un élément sémantique	27
7.27. Services de manipulation des dates	27
7.27.1. Formatage et date courante	27
7.27.2. Déterminer la différence temporelle entre deux dates	28
7.27.3. Déterminer la différence temporelle entre maintenant et la date de création ou de dernière modification d'un objet	28
7.27.4. Déterminer si une date précède ou suit une autre date	29
7.27.5. Déterminer si une propriété typée de type Date précède ou suit une autre	30
7.27.6. Addition et soustraction de temps sur une date	30
7.27.7. Calculer le temps qui s'est écoulé depuis la date décrite dans une propriété typée Date	31
7.28. Evaluer une requête AQL	31
7.29. Récupérer l'icone d'un élément sémantique	31
7.30. Récupérer une image de la bibliothèque d'images	32
8. Services du modules ArchiMate	33
8.1. Relations ArchiMate	33
8.2. Retrouver des éléments ArchiMate par leur nom	34
8.3. Retrouver un élément ArchiMate d'un type donné par son nom	34
8.4. Rechercher les éléments ArchiMate atteignables depuis un élément ArchiMate	36
8.5. Rechercher des chemins	37
9. Documentation officielle AQL	38

1. Introduction

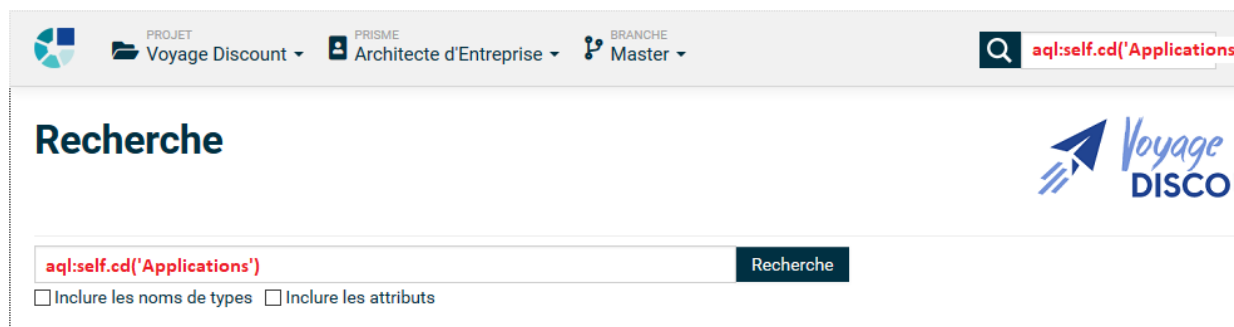
Il est possible de requêter le référentiel SmartEA via un langage spécifique : **AQL (Acceleo Query Language)**. Ce guide décrit son usage dans le cadre de SmartEA.

Note : Ce guide ne se substitue pas à la documentation officielle AQL. Vous pouvez retrouver les liens vers les documentations officielles en Section [Documentation officielle AQL](#).

AQL est utilisable dans :

- la recherche textuelle (aql:...),
- les widgets de la page d'accueil pour définir des camemberts, histogrammes, tableaux,
- les widgets des pages de détails,
- les templates de génération de diagrammes ArchiMate,
- les Smart Requests pour faire des exports au format Excel des données du référentiel,
- les générateurs de texte (Acceleo, M2Doc ou Java) pour exporter selon divers formats textuels,
- la vue Interpréteur de requêtes (modeleur uniquement),
- de nouveaux types de représentation (développement spécifique via SmartEA Developer).

Exemple de requête AQL utilisée pour faire une recherche dans le référentiel :



The screenshot shows the top navigation bar of the SmartEA interface with tabs for 'PROJET Voyage Discount', 'PRISME Architecte d'Entreprise', and 'BRANCHE Master'. A search bar on the right contains the query 'aql:self.cd('Applications')'. Below the navigation bar, the main area is titled 'Recherche' and features a large search input field containing the same query, a 'Recherche' button, and two checkboxes: 'Inclure les noms de types' and 'Inclure les attributs'.

Les requêtes s'appliquent toujours sur un objet. Cet objet est appelé l' **objet de contexte**.

Exemples :

- La racine du référentiel est l'objet de contexte
 - lors d'une recherche textuelle.
 - dans un widget de la page d'accueil.
- L'objet de contexte d'une Smart Request est l'objet sur lequel la Smart Request a été créée.
- L'objet sélectionné (dans l'explorateur de modèle ou une représentation) est l'objet de contexte lorsqu'on utilise la vue Interpréteur dans le modeleur.

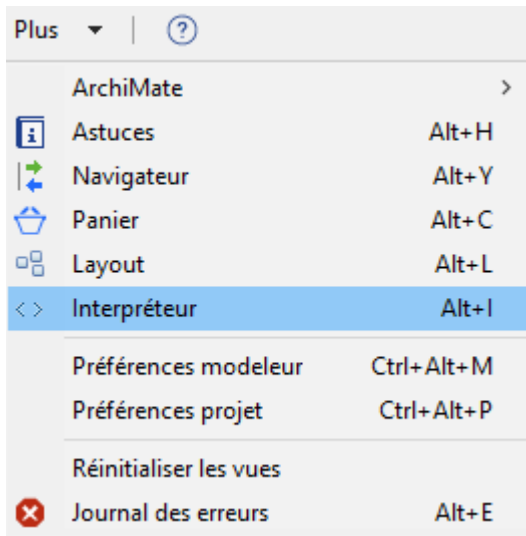
De nombreux services sont disponibles pour requêter le référentiel :

- Navigation dans les répertoires
- Manipulation des propriétés dynamiques
- Recherche d'éléments selon différents critères
- Etc.

La suite de ce document présente les services AQL fournis par défaut. Si vous souhaitez ajouter des services supplémentaires veuillez vous reporter au guide de développement SmartEA.

Pour mettre au point une requête AQL il est recommandé d'utiliser l'Interpréteur disponible dans le Modeleur.

Pour l'ouvrir, dépliez le menu «Plus» qui se trouve sur la barre de menu principal en haut du Modeleur.



Pour l'utiliser, cliquez sur un élément dans l'explorateur de modèle ou dans une représentation, puis entrez votre requête en la préfixant par `aql:` suivi de l'objet de contexte `self`.

Exemple :

```
aql:self.name
```

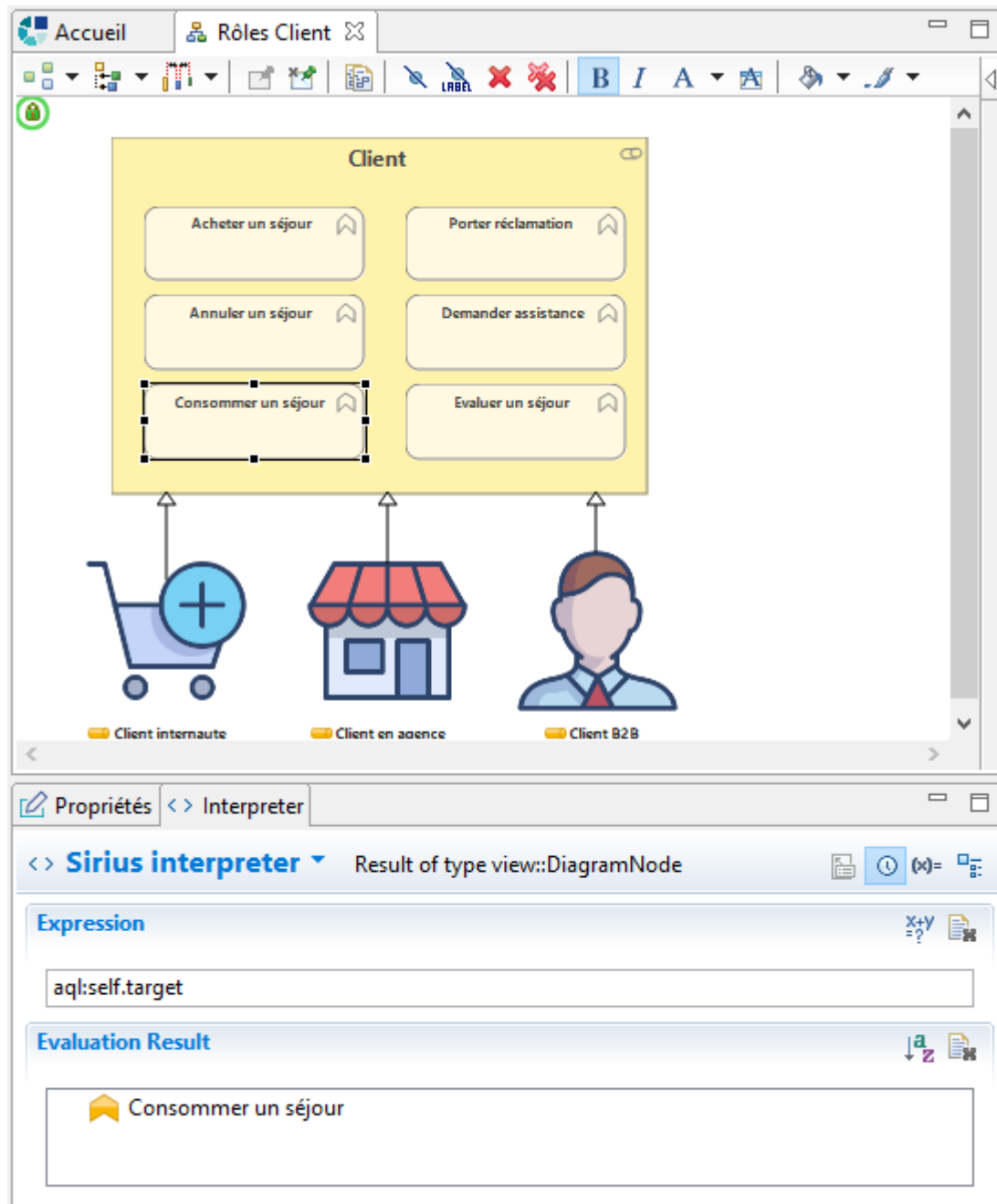
Cette requête renvoie la valeur de l'attribut `name` de l'objet `self`.

Attention : Pour utiliser l'Interpréteur en sélectionnant un élément sur un diagramme, vous devez utiliser la référence `target` pour traverser la couche graphique et accéder au modèle sémantique. En effet il est également possible de naviguer dans le modèle graphique avec AQL.

Dans la capture suivante, un élément est sélectionné dans un diagramme. La requête `aql:self` renvoie l'objet graphique.

The screenshot displays the Obeo SmartEA software interface. The top window, titled 'Rôles Client', shows a diagram of a 'Client' role. The diagram is a yellow rectangle containing six buttons: 'Acheter un séjour', 'Porter réclamation', 'Annuler un séjour', 'Demander assistance', 'Consommer un séjour', and 'Evaluer un séjour'. Below the diagram are three icons representing different client types: a shopping cart for 'Client internaute', a storefront for 'Client en agence', and a person for 'Client B2B'. Arrows point from each icon to the 'Consommer un séjour' button. The bottom window, titled 'Interpreter', shows the 'Sirius interpreter' results. The 'Expression' field contains 'aql:self'. The 'Evaluation Result' field shows a list containing 'Consommer un séjour'.

Dans la capture suivante, le même élément est sélectionné dans un diagramme mais la référence `target` est appelée. La requête `aql:self.target` renvoie ainsi l'objet sémantique.



Attention : Dans le cas d'une sélection dans un diagramme libre ArchiMate, pour traverser la couche graphique, vous devez utiliser la référence `target` puis `model` pour obtenir l'objet sémantique : `self.target.model`.

2. Requêtes simples

2.1. Premiers pas

Une requête AQL est préfixée par `aql:`.

L'élément de contexte sur lequel est lancé une requête est `self`.

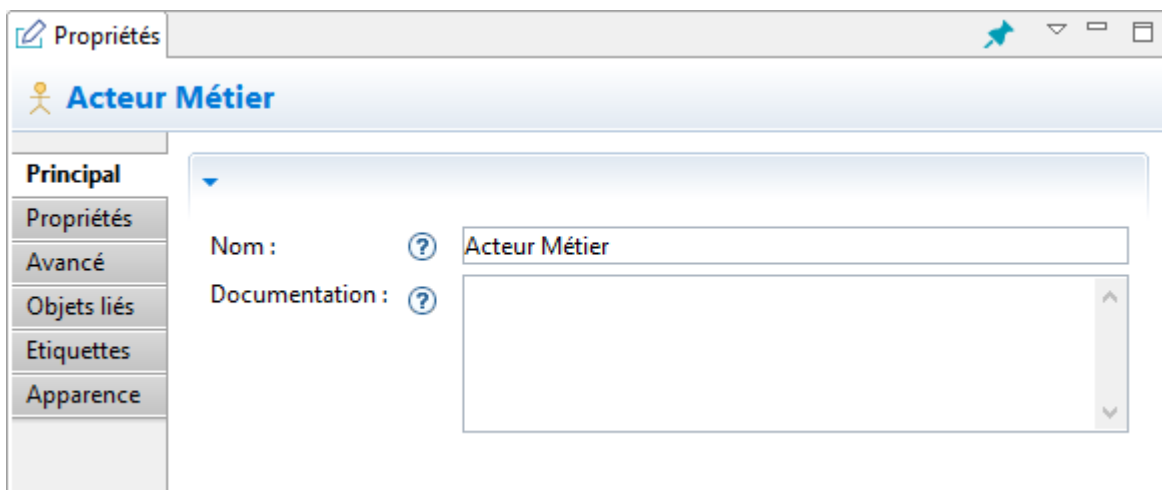
On accède aux attributs et références d'un élément avec l'opérateur `'.'` (point).

Exemple :

Si un objet sémantique de type `archimate::BusinessActor` nommé « Manager » est sélectionné dans l'explorateur de modèles alors la requête `aql:self.name` dans l'Interpréteur va renvoyer le nom de l'acteur sélectionné, c'est à dire Manager.

Tous les objets du référentiel sont conformes à un métamodèle. Les métamodèles définissent des concepts (Processus, Acteur, Composant Technologique, etc.), des attributs sur ces concepts (nom, description, etc.) et des références entre ces concepts.

Par exemple, tous les concepts du métamodèle ArchiMate possèdent deux attributs : `name` et `documentation`.



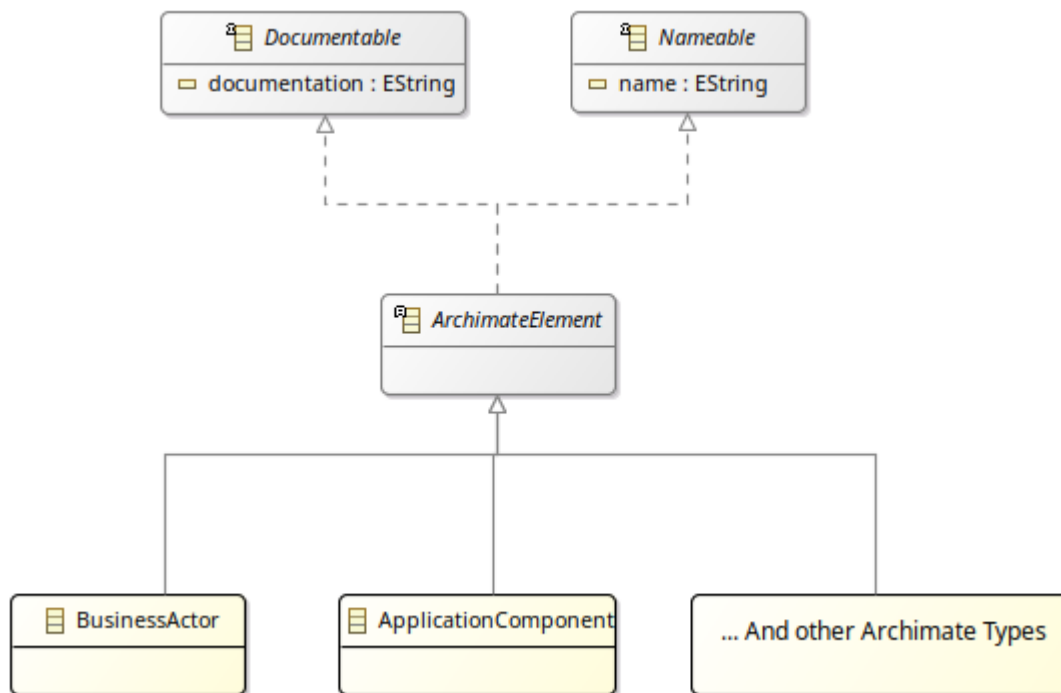
Ainsi pour obtenir le nom d'un élément ArchiMate il suffit de faire :

```
aql:self.name
```

De la même manière pour récupérer la valeur du champ documentation d'un élément ArchiMate il suffit de faire :

```
aql:self.documentation
```

Cela se traduit au niveau du métamodèle de la façon suivante :



Le type abstrait `ArchimateElement`, permet de manipuler les éléments de manière générique. Les attributs `name` et `documentation` sont hérités par tout type `ArchiMate` concret.

2.2. Appel de services

SmartEA fournit un ensemble de services détaillés dans la suite de ce document. Vous avez également la possibilité de contribuer vos propres services (développement nécessaire).

Les services sont utilisables de la même façon qu'un attribut ou référence, mais avec des parenthèses permettant de préciser les paramètres si nécessaire.

Dans la suite du document les services sont décrits de la manière suivante :

```
TypeDuContexte.nomDuService(TypeDuParamètre_1 paramètre_1, ...) : TypeDuRésultat
```

- `TypeDuContexte` : le type des objets sur lequel s'applique le service. Le type d'un objet est, en termes simples, la catégorie de cet objet. Par exemple, dans la notation `ArchiMate`, `SystemSoftware` et `ApplicationComponent` sont les types des objets de catégorie système logiciel et composant applicatif, respectivement. On trouvera également des catégories plus générique comme `Object` qui comporte tous les objets ou `PropertyContainer` qui comporte tous les objets capables de porter des propriétés dynamiques.
- `nomDuService` : le nom du service.
- `TypeDuParamètre_n` : le type du paramètre numéro `n`.
- `paramètre_n` : le nom du paramètre numéro `n`.
- `TypeDuRésultat` : le type du résultat.

Exemple :

`aql:self.getLabel()` renvoie une chaîne de caractères, le label de l'élément `self`.

3. Manipulation des chaînes de caractères

Si dans vos requêtes AQL vous êtes amenés à manipuler des chaînes de caractères, AQL propose des services :

- `String.toLowerCase() : String`
- `String.toLowerCaseFirst () : String`
- `String.toUpperCase() : String`
- `String.toUpperCaseFirst () : String`
- `String.concat(String s) : String`
- `String.size() : Integer`
- `String.substring(Integer lower, Integer upper) : String`
- `String.contains(String substring) : Boolean`
- `String.equalsIgnoreCase(String other) : Boolean`
- `String.startsWith(String substring) : Boolean`
- ...

Exemple :

```
aql:self.name.contains('Mana')
```

Cette requête renvoie `true` si `self` est un objet dont l'attribut `name` est «Manager».

```
aql:self.name.startsWith('Manager')
```

Cette requête renvoie `true` si `self` est un objet dont l'attribut `name` est «Manager».

Elle renvoie `false` si `self` est un objet dont l'attribut `name` est «Le Manager Principal».

Pour une description plus détaillée, référez vous à la documentation officielle AQL. Les liens sont donnés en Section [Documentation officielle AQL](#).

4. Opérations booléennes

Il est possible d'utiliser des opérateurs booléens dans les requêtes AQL. Les opérateurs booléens sont :

- and
- or
- xor
- not
- implies

Exemple :

```
aql:self.name.contains('Manager') and self.name.contains('Principal')
```

Cette requête renvoie `true` si `self` est un objet dont l'attribut `name` est «Manager Principal».

Elle renvoie `false` si `self` est un objet dont l'attribut `name` est «Manager».

```
aql:self.name.contains('Manager') or self.name.contains('Principal')
```

Cette requête renvoie `true` si `self` est un objet dont l'attribut `name` est «Manager Principal».

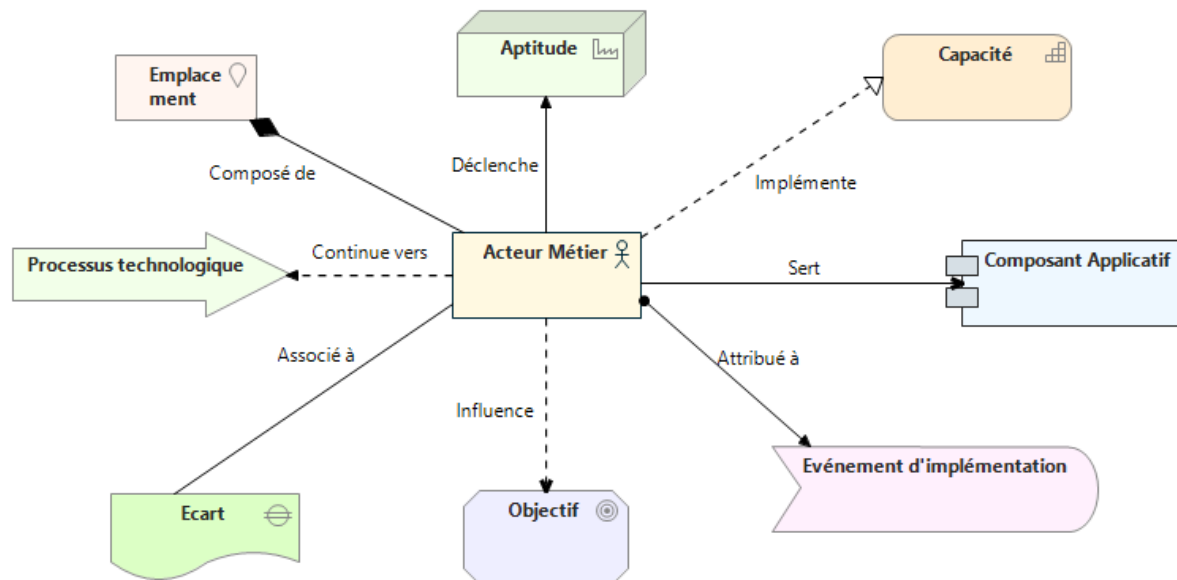
Elle renvoie également `true` si `self` est un objet dont l'attribut `name` est «Manager».

Pour une description plus détaillée, référez vous à la documentation officielle AQL. Les liens sont donnés en Section [Documentation officielle AQL](#).

5. Manipulation des collections

Les éléments d'un modèle sémantique sont reliés entre eux par des références. Ces références peuvent faire le lien entre un objet et plusieurs objets. Dans ce cas une requête sur la référence renvoie non pas un objet mais une collections d'objets.

La figure suivante montre le métamodèle ArchiMate où on peut voir que le concept d'Acteur Métier a un ensemble de références vers d'autres concepts. Toutes ces références sont multivaluées.



Ainsi, dans ce contexte, l'appel à la référence `serve ( )` sur un Acteur Métier renvoie la collection des éléments reliés à l'élément par la relation de servitude.

Pour effectuer une opération sur un élément simple on utilise l'opérateur `'.'` comme vu précédemment.

Pour une collection, on utilise le symbole `'->'` (un tiret suivi du signe supérieur).

Exemple :

```
aql:self.elements->size()
```

Si `self` est un répertoire cette requête renvoie le nombre d'éléments dans la Collection résultat et donc le nombre d'éléments dans le répertoire.

Ainsi divers services sont disponibles sur les collections (`->`) :

- `isEmpty ()` : Boolean
- `size ()` : Integer
- `select (OclExpression exp)` : Collection (T)
- `reject (OclExpression exp)` : Collection (T)
- `sortedBy (OclExpression exp)` : Sequence (T)
- `sep (String separator)` : Sequence (OclAny)
- `filter (OclType type)` : Collection (OclType)
- ...

L'opérateur `'.'` appliqué sur une collection donne accès aux opérations et attributs du type des éléments de la collection.

Exemple :

```
aql:self.elements.name
```

Si `self` est un répertoire cette requête renvoie la liste des noms des éléments contenus dans le répertoire.

Pour une description plus détaillée, référez vous à la documentation officielle AQL. Les liens sont donnés en Section [Documentation officielle AQL](#).

6. Services génériques

6.1. Services sur le type

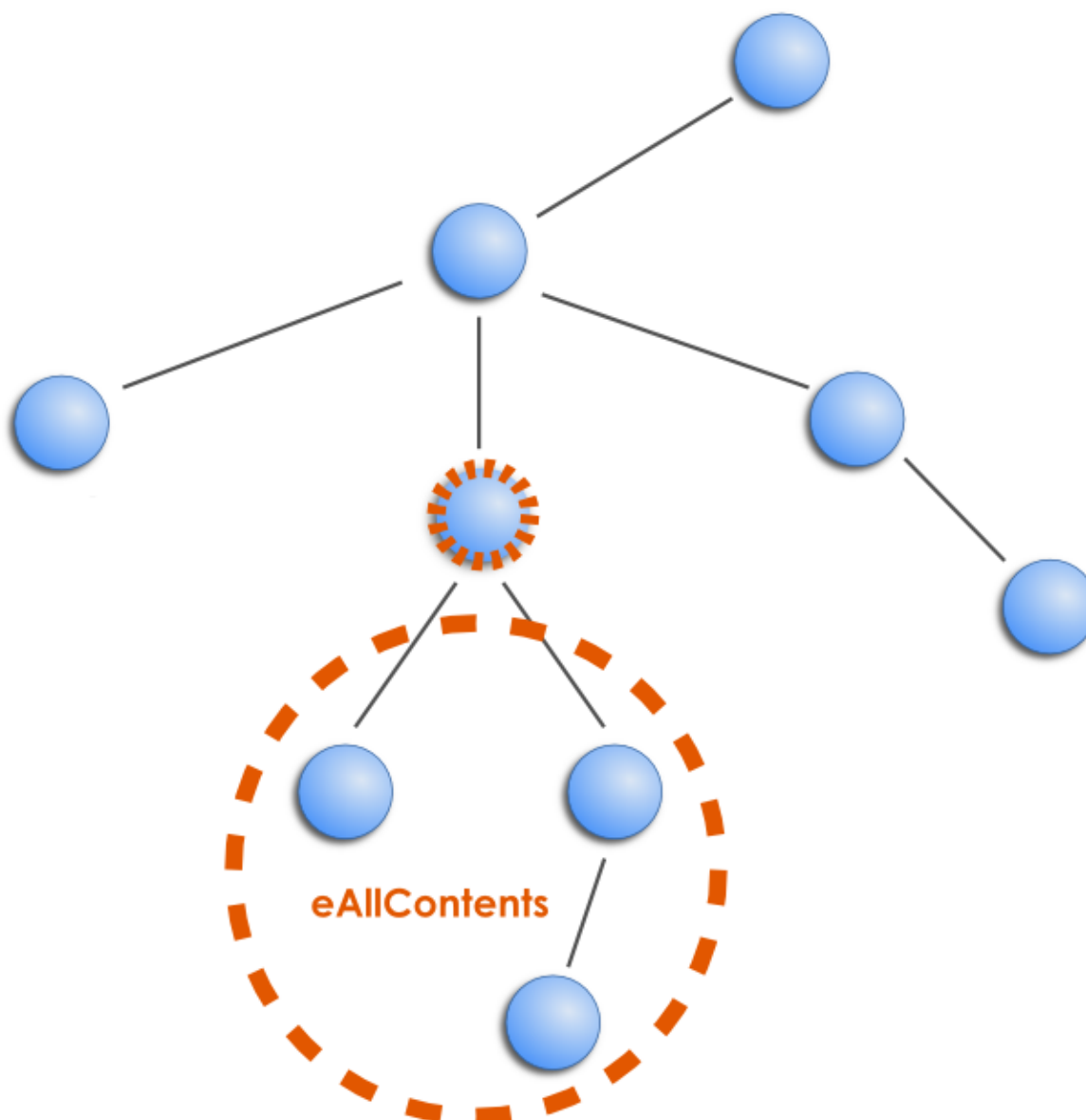
Tous les éléments disposent des mêmes opérations génériques permettant des opérations sur le type de l'objet courant :

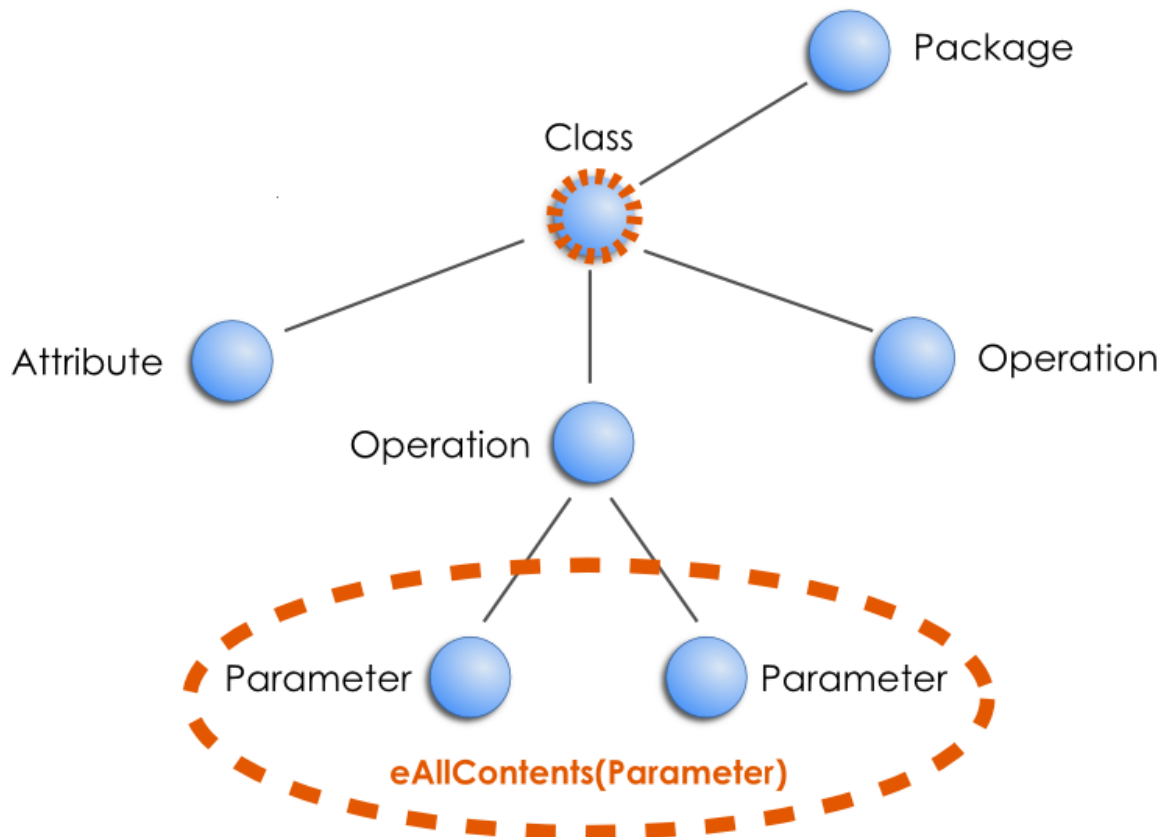
- `oclAsType` (Classifieur c) : T
 - permet de caster l'objet de contexte.
- `oclIsKindOf` (Classifieur c) : Boolean
 - permet de savoir si l'objet de contexte est d'un type donné ou d'un super type du type donné.
- `oclIsTypeOf` (Classifieur c) : Boolean
 - permet de savoir si l'objet de contexte est d'un type donné.

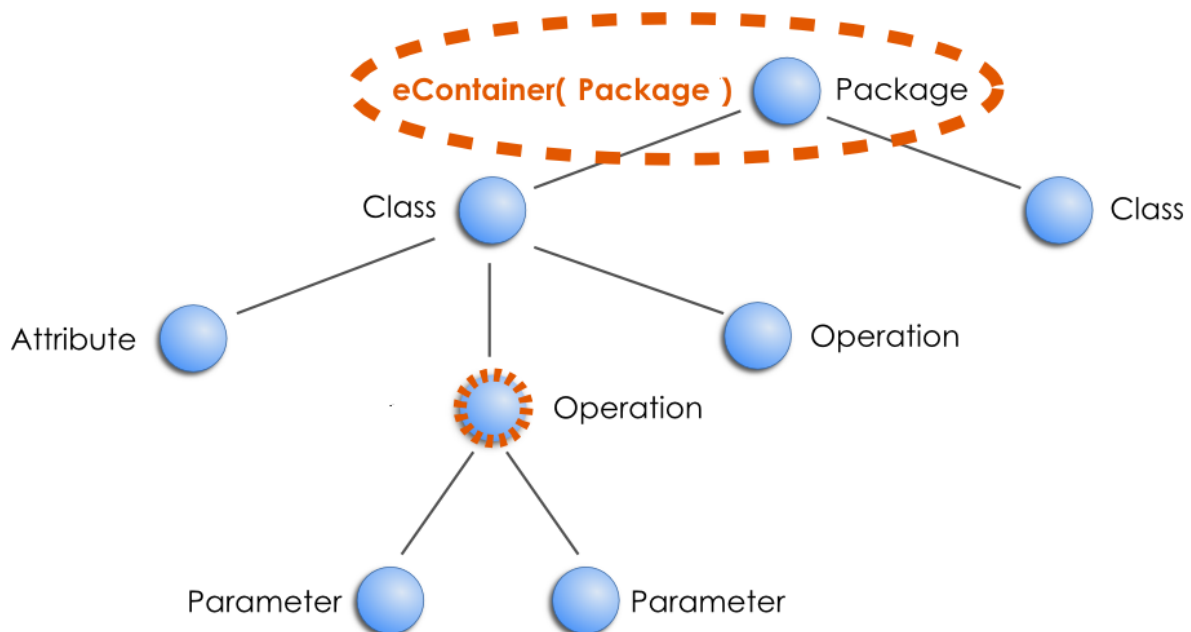
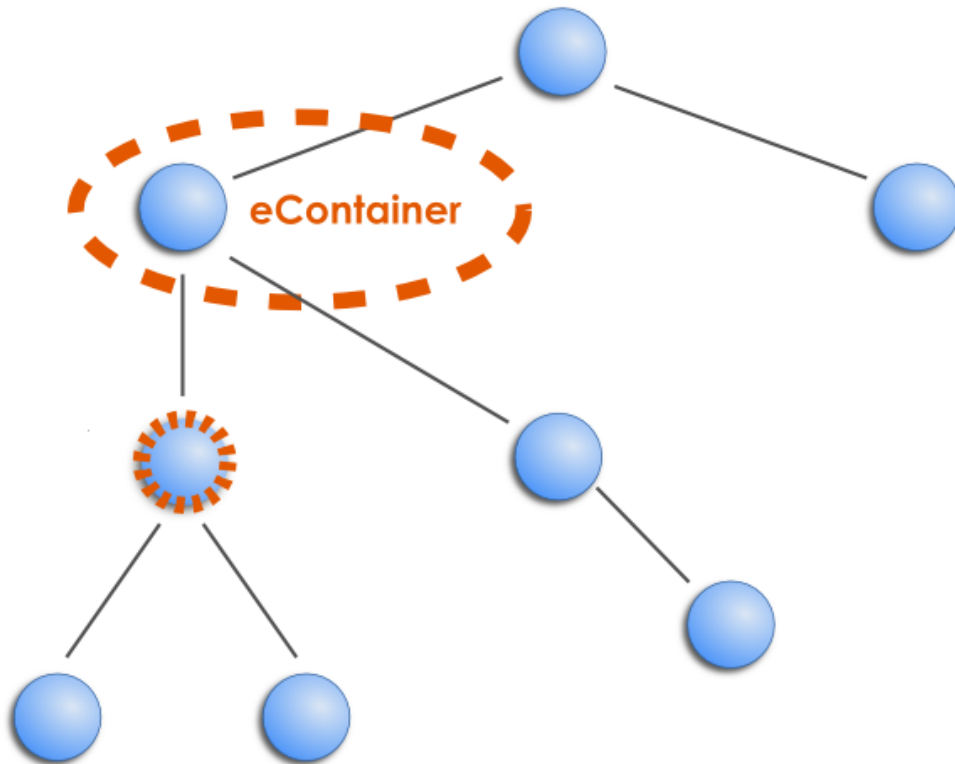
Pour une description plus détaillée, référez vous à la documentation officielle AQL. Les liens sont donnés en Section [Documentation officielle AQL](#).

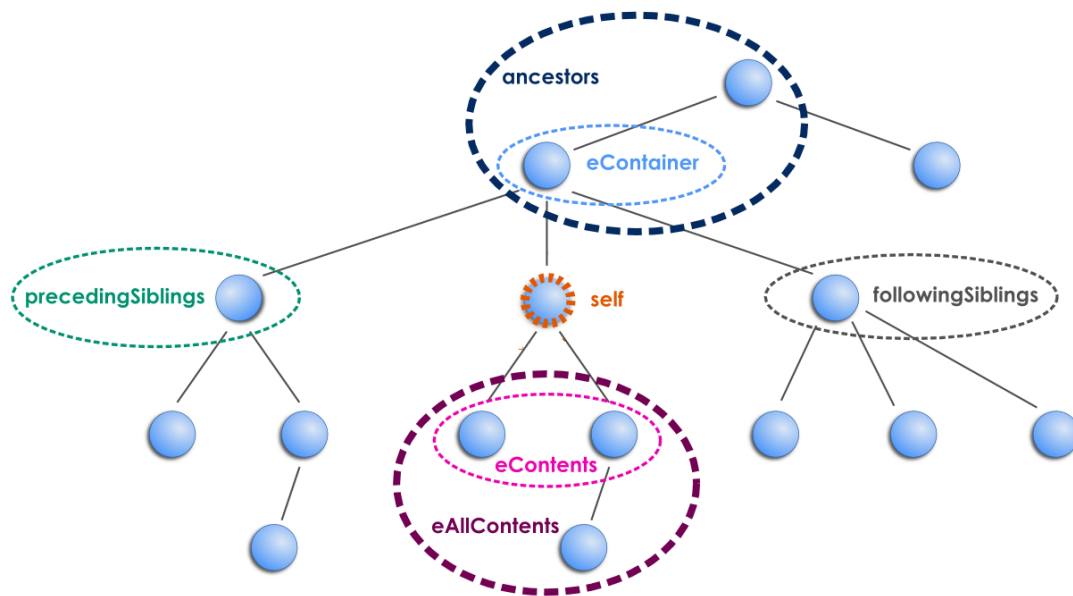
6.2. Services de navigation

Tous les éléments disposent également de services de navigation. Les figures suivantes résument les services disponibles.







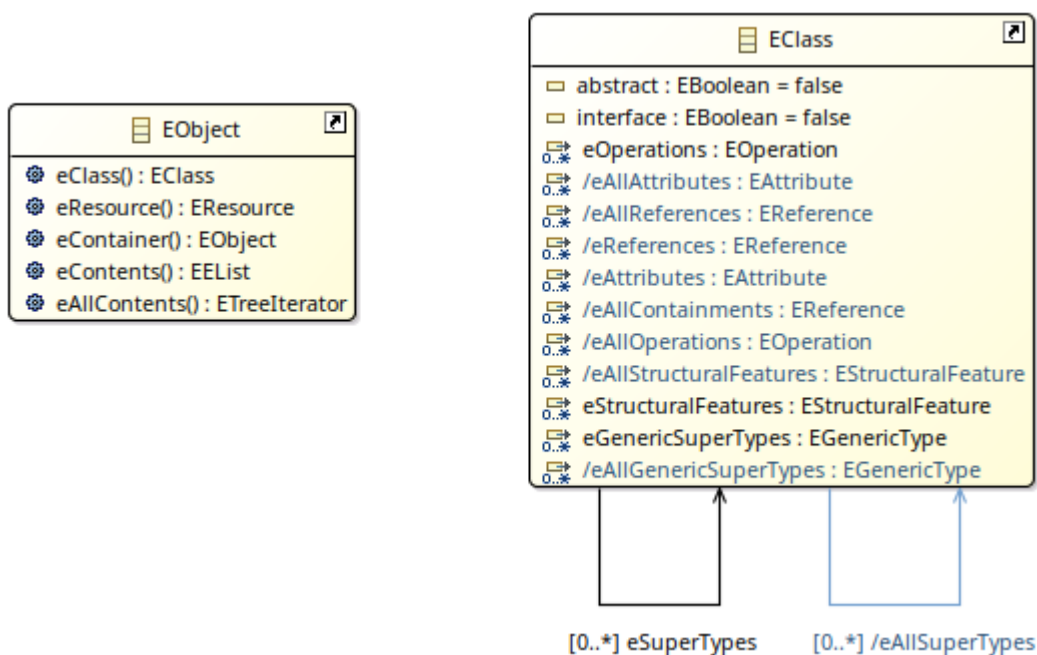


Pour une description plus détaillée, référez vous à la documentation officielle AQL. Les liens sont donnés en Section [Documentation officielle AQL](#).

6.3. Services sur EObject

Tous les services de la section précédente sont disponibles sur tous les objets du référentiel car ils sont portés par le super type abstrait de tous les objets du référentiel : EObject.

D'autres services que ceux présentés dans les figures précédentes sont disponibles comme le montre la figure suivante.



Par exemple, la métaclasse d'un élément peut être obtenue avec la requête suivante :

```
aql:self.eClass()
```

Les attributs et relations d'un élément peuvent ainsi être découverts de la manière suivante :

```
aql:self.eClass().eAllStructuralFeatures
```

Pour une description plus détaillée, référez vous à la documentation officielle AQL. Les liens sont donnés en Section [Documentation officielle AQL](#).

Vous trouverez également ici une [carte de référence Ecore](https://cedric.brun.io/images/blog/2023/Ecore_reference_card.pdf) (https://cedric.brun.io/images/blog/2023/Ecore_reference_card.pdf).

7. Services SmartEA

Cette section liste les services apportés par SmartEA au dessus d'AQL.

7.1. Retrouver le label d'un objet

Le service `getLabel` renvoie le label d'un objet quelconque.

```
Object.getLabel() : String
```

Exemple :

`self.getLabel()` appliqué sur un répertoire nommé **Applications** renvoie la chaîne de caractères **Applications**.

7.2. Naviguer dans les répertoires.

Le service `cd` permet de naviguer dans la hiérarchie de répertoires à partir de la racine du référentiel. La finalité de cette navigation est de positionner le contexte dans un nouveau répertoire afin de pouvoir y faire une requête.

```
Resource.cd(String path) : Folder
```

Exemple :

`self.cd('Marketing/Offres/Gestion des partenaires')` appliqué à la racine du référentiel renvoie le répertoire **Gestion des partenaires** qui se trouve dans le répertoire **Offres** qui se trouve lui même dans le répertoire **Marketing**.

Le service suivant rend le même service que le service précédent mais au lieu de s'appliquer sur la racine du référentiel, il s'applique sur un répertoire particulier.

```
Folder.cd(String path) : Folder
```

Exemple :

`self.cd('Offres/Gestion des partenaires')` appliqué sur le répertoire **Marketing** renvoie le répertoire **Gestion des partenaires** qui se trouve dans le répertoire **Offres** qui se trouve lui même dans le répertoire **Marketing**.

7.3. Retrouver le chemin absolu d'un répertoire

Le service `fullpath` renvoie le chemin absolu d'un répertoire.

```
Folder.fullpath() : String
```

Exemple :

`self.fullpath()` appliqué sur le répertoire **Gestion des partenaires** renvoie la chaîne des caractères **Marketing/Offres/Gestion des partenaires**.

7.4. Retrouver le parent d'un répertoire

Le service `parentFolder` appliqué sur un objet quelconque du référentiel renvoie le répertoire contenant l'objet.

```
Object.parentFolder() : Folder
```

Exemple :

`aql:self.parentFolder()` appliqué sur l'acteur **Equipe Offre** renvoie le répertoire **Offres**.

7.5. Retrouver un ensemble d'éléments d'un ou plusieurs types donnés

Appliqué sur un objet quelconque, le service `childrenByType` renvoie tous les objets enfants (liés par une relation de contenance) de cet objet et du type donné en paramètre. Le type donné en paramètre peut être une expression régulière.

```
Object.childrenByType(String typeName) : Set<Object>
```

Exemple :

`aql:self.childrenByType('Actor|Role')` appliqué sur un répertoire renvoie l'ensemble des objets de type **Acteur** ou **Rôle** contenus dans le répertoire ou ses sous répertoires.

Appliqué sur une collection d'objets, le service `elementsByType` filtre les objets de la collection selon le type donné en paramètre. Le type donné en paramètre peut être une expression régulière.

```
Collection<Object>->elementsByType(String typeName) : Set<Object>
```

Exemple :

`aql:self.childrenByType('Actor|Role')->elementsByType('Actor')` renvoie l'ensemble des objets de type **Acteur**.

7.6. Retrouver un ensemble d'objets ayant une propriété dynamique donnée

Appliqué sur un objet quelconque, le service `childrenByProperty` renvoie tous les objets enfants (liés par une relation de contenance) de cet objet et ayant une propriété dynamique donnée. Le nom exact de la propriété dynamique doit être donné.

```
Object.childrenByProperty(String propertyName) : Set<PropertiesContainer>
```

Exemple :

`aql:self.childrenByProperty('Type de licence')` appliqué sur un répertoire renvoie l'ensemble des objets du répertoire et de ses sous répertoires qui ont une propriété dynamique **Type de licence**.

Appliqué sur une collection d'objets, le service `elementsByProperty` ne retient que les objets de la collection qui ont une propriété dynamique donnée.

```
Collection<PropertiesContainer>->elementsByProperty(String propertyName) :  
Set<PropertiesContainer>
```

Exemple :

`aql:self.childrenByProperty('Type de licence')->elementsByProperty('Fin maintenance')` filtre les objets qui ont une propriété dynamique **Fin maintenance**.

7.7. Retrouver un ensemble d'objets ayant une propriété dynamique donnée avec une valeur donnée

Appliqué sur un objet quelconque, le service `childrenByProperty` renvoie tous les objets enfants (liés par une relation de contenance) de cet objet et ayant une propriété dynamique donnée avec une valeur donnée.

Le nom exact de l'attribut dynamique doit être donné.

La valeur de cette propriété dynamique peut être exprimée sous forme d'expression régulière. Dans son expression la plus simple, une valeur de propriété complète peut être utilisée.

```
Object.childrenByProperty(String propertyName, String propertyValue) :  
Collection<PropertiesContainer>
```

Exemple :

`aql:self.childrenByProperty('Type de licence', 'commerc.*')` appliqué sur un répertoire renvoie l'ensemble des objets du répertoire et de ses sous répertoires qui ont une propriété dynamique **Type de licence** avec la valeur **commerc.***.

Appliqué sur une collection d'objets, le service `elementsByProperty` ne retient que les objets de la collection qui ont une propriété dynamique donnée avec une valeur donnée. Le nom exact de l'attribut dynamique doit être donné. La valeur de cette propriété dynamique peut être exprimée sous forme d'expression régulière.

```
Collection<PropertiesContainer>->elementsByProperty(String propertyName, String  
    propertyValue) : Set<PropertiesContainer>
```

Exemple :

`aql:self.childrenByProperty('Type de licence', 'commerc*')->elementsByProperty('Fin maintenance', '2019|2020')` filtre les objets qui ont une propriété dynamique **Fin maintenance** avec la valeur **2019** ou **2020**.

7.8. Retrouver la valeur d'une propriété dynamique donnée

Le service `getPropertyValue` renvoie la valeur d'une propriété dynamique donnée.

```
PropertiesContainer.getPropertyValue(String propertyName) : String
```

Exemple :

`aql:self.getPropertyValue('Type de licence')` renvoie la valeur de la propriété dynamique **Type de licence** si celle-ci existe.

7.9. Retrouver une propriété dynamique donnée.

Le service `getProperty` renvoie la propriété dynamique correspondant à nom donné.

```
PropertiesContainer.getProperty(String propertyName) : Property
```

Exemple :

`aql:self.getPropertyValue('Type de licence')` renvoie la propriété dynamique **Type de licence** si celle-ci existe.

7.10. Retrouver la valeur d'une propriété dynamique

Le service `getStringValue` renvoie la valeur d'une propriété dynamique sous forme de chaîne de caractères.

```
String getStringValue(Property property)
```

Exemple :

`aql:self.getStringValue()` renvoie la valeur de l'objet de contexte si celui-ci est une propriété dynamique.

7.11. Tester la présence d'une propriété dynamique donnée

Le service `propertyDefined` permet de tester si une propriété d'un nom donné est définie une et une seule fois sur l'objet cible. Si la propriété est définie plus d'une fois ou n'est pas définie alors le service renvoie faux. Il renvoie vrai si la propriété est définie une seule fois dans l'objet.

```
PropertiesContainer.propertyDefined(String name) : boolean
```

Exemple :

`aql:self.propertyDefined('status')` renvoie `vrai` si la propriété **status** est définie de manière unique sur l'objet référencé par `self` et `faux` dans les autres cas.

7.12. Tester la correction d'une définition de propriété donnée vis-à-vis d'une liste de valeur

Le service `correctProperty` permet de tester qu'une propriété donnée par son nom est définie de manière unique sur l'objet cible de l'appel et possède une valeur dans la liste de valeurs spécifiée.

```
PropertiesContainer.correctProperty(String name, String values) : boolean
```

Les valeurs sont spécifiées comme une liste de chaînes de caractères séparées par des virgules.

Exemple :

`aql:self.correctProperty('status', 'brouillon,complet,validé')` renvoie `vrai` si la propriété **status** est définie de manière unique sur l'objet référencé par **self** et possède une valeur dans la liste spécifiée («*brouillon*», «*complet*», «*validé*»). Le service renvoie `faux` dans les autres cas.

7.13. Tester si une valeur est conforme à la contrainte d'une propriété typée

Le service `validateValue` appliqué sur une propriété typée permet de vérifier qu'une valeur est conforme ou non à la contrainte définie sur la propriété typée.

```
TypedProperty.validateValue(String value) : Boolean
```

Exemple :

Si une propriété typée de type `String` est définie avec une contrainte précisant que la valeur de la propriété doit être formée de 3 caractères, alors :

- `aql:self.validateValue('ABC')` renvoie `vrai`,
- `aql:self.validateValue('ABCD')` renvoie `faux`.

7.14. Tester qu'une contrainte est applicable sur un type de propriété typée

Le service `validateConstraint` appliqué sur une chaîne de caractères permet de vérifier que cette chaîne de caractères est applicable sur un type de propriété typée passé en paramètre.

```
String.validateConstraint(String typeLiteral) : Boolean
```

Exemple :

- `aql:'self > 100 and self<200'.validateConstraint('INTEGER')` renvoie `vrai`.
- `aql:'self > 100 ET self<200'.validateConstraint('INTEGER')` renvoie `faux` (ET n'est pas reconnu).

7.15. Transformer un type de propriété typée en chaîne de caractères

Les services `typeToString` et `stringToType` permettent d'obtenir le type d'un `TypedProperty` sous forme de `String` et d'obtenir à partir d'un `String` un Type de propriété typée.

```
TypedProperty.typeToString() : String
```

```
String.toStringType() : Type
```

7.16. Vérifier qu'une propriété typée est d'un certain type

Des services sont disponibles pour tester le type d'une propriété typée.

Tester si une propriété typée est de type numérique : entier ou réel.

```
TypedProperty.isNumeric() : Boolean
```

Tester si une propriété typée est de type énumération.

```
TypedProperty.isEnum() : Boolean
```

Tester si une propriété typée est de type booléen.

```
TypedProperty.isBoolean() : Boolean
```

Tester si une propriété typée est de type texte : chaîne de caractères ou chaîne de caractères multi-lignes.

```
TypedProperty.isText() : Boolean
```

Tester si une propriété typée est de type date.

```
TypedProperty.isDate() : Boolean
```

7.17. Retrouver un objet à partir de son identifiant logique

Les services `toObject` et `toArtifact` renvoient l'objet et l'artefact correspondant à un identifiant logique.

```
String.toObject() : Object
```

```
String.toArtifact() : Artifact
```

7.18. Retrouver les artefacts associés à un objet

Le service `getArtifacts` appliqué sur un objet quelconque du référentiel renvoie les artefacts associés.

```
Object.getArtifacts() : List<Artifact>
```

Le service `getSiriusArtifacts` appliqué sur un objet quelconque du référentiel renvoie les artefacts associés ayant un identifiant de point de vue et de représentation Sirius donnés.

```
Object.getSiriusArtifacts(String viewpointID, String representationID) : List<Artifact>
```

7.19. Retrouver les artefacts où un objet apparaît

Le service `getAllArtifacts` appliqué sur un objet quelconque du référentiel renvoie les artefacts où apparaît l'objet.

```
Object.getAllArtifacts() : List<Artifact>
```

Trois services sont également disponibles pour filtrer les résultats obtenus :

```
CD0Object.getAllArtifactsMatchingName(String nameRegex) : List<Artifact>
```

Ce service renvoie les artefacts Sirius où apparaît l'objet et ces artefacts ont un nom qui respecte le pattern **nameRegex**.

```
CD0Object.getAllArtifactsByType(String viewpointId, String typeId) : List<Artifact>
```


Ce service renvoie les artefacts Sirius où apparaît l'objet et ces artefacts ont pour identifiant de point de vue **viewpointId** et pour identifiant de représentation **typeId**.

```
CD00object.getAllArtifactsByType(String typeId) : List<String>
```

Ce service renvoie les artefacts Sirius où apparaît l'objet et ces artefacts ont pour identifiant de représentation **typeId**.

Des services similaires sont également disponibles pour récupérer non pas les artefacts mais les identifiants des artefacts :

```
CD00object.getAllArtifactsIds(CD00object context) : List<String>
```

Ce service renvoie les identifiants de tous les artefacts Sirius où apparaît l'objet.

```
CD00object.List<String> getAllArtifactsIdsMatchingName(String nameRegex) : List<String>
```

Ce service renvoie les identifiants des artefacts Sirius où apparaît l'objet et ces artefacts ont un nom qui respecte le pattern **nameRegex**.

```
CD00object.getAllArtifactsIdsByType(String viewpointId, String typeId) : List<String>
```

Ce service renvoie les identifiants des artefacts Sirius où apparaît l'objet et ces artefacts ont pour identifiant de point de vue **viewpointId** et pour identifiant de représentation **typeId**.

```
CD00object.List<String> getAllArtifactsIdsByType(String typeId) : List<String>
```

Ce service renvoie les identifiants des artefacts Sirius où apparaît l'objet et ces artefacts ont pour identifiant de représentation **typeId**.

7.20. Tester si un élément est stéréotypé par un stéréotype particulier

Les services `hasStereotypeWithId` et `hasStereotypeWithName` appliqués sur un objet stéréotypable du référentiel renvoient vrai si l'objet est stéréotypé par un stéréotype ayant un certain identifiant ou nom de stéréotype.

```
Stereotypable.hasStereotypeWithId(String name) : Boolean
```

```
Stereotypable.hasStereotypeWithName(String name) : Boolean
```

Exemple :

`aql:self.hasStereotypeWithName('Suite Applicative')` appliqué sur un objet `Stereotypable` renvoie vrai si l'objet est stéréotypé par le stéréotype de nom `Suite Applicative`.

7.21. Retrouver les éléments sémantiques et les artefacts liés à un artefact donné.

Les services `getSemanticElements` et `getLinkedArtifacts` appliqués à un artefact vont retourner la liste des éléments sémantiques représentés dans l'artefact pour le premier et la liste des artefacts liés pour le second. Un paramètre additionnel peut être donné pour inclure les éléments cachés de l'artefact dans les résultats.

```
Artifact.getSemanticElements() : List<Object>
```

```
Artifact.getSemanticElements(Boolean includeHidden) : List<Object>
```

```
Artifact.getLinkedArtifacts() : List<Artifact>
```

```
Artifact.getLinkedArtifacts(Boolean includeHidden) : List<Artifact>
```

7.22. Récupérer l'identifiant du projet courant

Le service `getProjectId` appliqué sur un objet quelconque renvoie l'identifiant du projet courant.

```
Object.getProjectId(CD00Object any) : String
```

7.23. Récupérer l'identifiant de la branche courante

Le service `getBranchId` appliqué sur un objet quelconque renvoie l'identifiant de la branche courante.

```
Object.getBranchId(CD00Object any) : String
```

7.24. Récupérer le prisme courant

Le service `currentPrism` appliqué sur un objet quelconque renvoie le prisme courant.

```
Object.currentPrism() : String
```

7.25. Récupérer l'url d'un élément sémantique

Plusieurs services sont disponibles pour récupérer l'url d'un élément sémantique :

```
CD00Object.makeHTMMLink() : String
```

Ce service renvoie le lien HTML correspondant à `object` et ayant la forme suivante `<a style='text-decoration:underline' href='...'>Label de l'objet</a>`. Le lien est un lien pour le projet courant, la branche courante et le prisme courant.

```
CD00Object.makeHTMMLink(String prismName) : String
```

Ce service renvoie le lien HTML correspondant à `object` et ayant la forme suivante `<a style='text-decoration:underline' href='...'>Label de l'objet</a>`. Le lien est un lien pour le projet courant, la branche courante et le prisme donné en paramètre.

```
CD00Object.makeHTMMLink(String prismName, String linkText) : String
```

Ce service renvoie le lien HTML correspondant à `object` et ayant la forme suivante `<a style='text-decoration:underline' href='...'>Texte donné par le paramètre linkText</a>`. Le lien est un lien pour le projet courant, la branche courante et le prisme courant.

```
CD00Object.makeHTMMLink(String branchName, String prismName, String linkText) : String
```

Ce service renvoie le lien HTML correspondant à `object` et ayant la forme suivante `<a style='text-decoration:underline' href='...'>Texte donné par le paramètre linkText</a>`. Le lien est un lien pour le projet courant, la branche donnée en paramètre et le prisme donné en paramètre.

```
CD00Object.makeHTMMLink(String projectName, String branchName, String prismName, String linkText) : String
```

Ce service renvoie le lien HTML correspondant à `object` et ayant la forme suivante `<a style='text-decoration:underline' href='...'>Texte donné par le paramètre linkText</a>`. Le lien est un lien pour le projet donné en paramètre, la branche donnée en paramètre et le prisme donné en paramètre.

```
CD00Object.makeLink() : String
```

Ce service renvoie l'URL correspondant à `object`. L'URL correspond au projet courant, à la branche courante et au prisme courant.

```
CD00Object.makeLink(String prismName) : String
```

Ce service renvoie l'URL correspondant à `object`. L'URL correspond au projet courant, à la branche courante et au prisme donné en paramètre.

```
CD00object.makeLink(String branchName, String prismName) : String
```

Ce service renvoie l'URL correspondant à `object`. L'URL correspond au projet courant, à la branche et au prisme donnés en paramètre.

```
CD00object.makeLink(String projectName, String branchName, String prismName) : String
```

Ce service renvoie l'URL correspondant à `object`. L'URL correspond au projet donné en paramètre, à la branche donnée en paramètre et au prisme donné en paramètre.

Exemple :

```
aql:self.makeHTMLink()
```

Appliquée sur un objet sémantique cette requête renvoie :

```
<a style='text-decoration:underline' href='/smartea/voyagediscount/Master/${currentPrismID}/app/object/417d210a'>CRM</a>
```

Appliquée sur le même objet que précédemment la requête suivante :

```
aql:self.makeLink()
```

renvoie

```
/smartea/voyagediscount/Master/EntArch/app/object/417d210a
```

7.26. Récupérer les informations de traçabilité d'un élément sémantique

Quatre services sont disponibles pour récupérer les informations de traçabilité d'un objet sémantique.

```
CD00object.getCreationTimestamp() : String
```

Ce service renvoie la date de création de l'objet sémantique (selon le pattern défini dans les préférences projet).

```
CD00object.getCreationUser() : String
```

Ce service renvoie l'identifiant l'utilisateur ayant créé l'objet sémantique.

```
CD00object.getLastModificationTimestamp() : String
```

Ce service renvoie la date de dernière modification de l'objet sémantique (selon le pattern défini dans les préférences projet).

```
CD00object.getLastModificationUser() : String
```

Ce service renvoie l'identifiant du dernier utilisateur ayant modifié l'objet sémantique.

7.27. Services de manipulation des dates

7.27.1. Formatage et date courante

Les services `getDefaultDateFormat` et `getTraceabilityDateFormat` renvoient le formatage par défaut des dates configuré au niveau du serveur et le formatage par défaut utilisé pour les informations de traçabilité.

```
Object.getDefaultDateFormat() : String
```

Permet de récupérer le format de date global de la configuration du serveur SmartEA.

Ce service s'applique sur n'importe quel objet.

```
Object.getTraceabilityDateFormat() : String
```

Permet de récupérer le format de date de la tracabilité des préférences projet.

Ce service s'applique sur n'importe quel objet.

Le service date permet de récupérer la date actuelle dans un format donné.

```
String.date() : String
```

Ce service s'applique sur une chaîne de caractère spécifiant le formatage souhaité (yyyy, MM, dd, etc.).

Ce service invalide le cache serveur d'une requête utilisant ce service.

Exemples :

`self.getDefaultDateFormat().date()` donne la date actuelle dans le format par défaut configuré sur le serveur SmartEA.

`'dd/MM/yyyy'.date()` donne par exemple pour résultat `<10/09/2023>` si la date courante est 10 septembre 2023.

7.27.2. Déterminer la différence temporelle entre deux dates

Le service `between` permet de calculer la différence temporelle entre deux dates.

```
String.between(String startDate, String startDateFormat, String endDate, String endDateFormat) : String
```

Ce service s'applique sur une chaîne de caractères spécifiant l'unité temporelle dans laquelle le résultat est souhaité. Les termes autorisés sont :

- years
- months
- weeks
- days
- hours
- minutes
- seconds

Le premier paramètre de ce service est la date de début. Le second paramètre est le format de la date de début. Le troisième paramètre est la date de fin. Le quatrième paramètre est le format de la date de fin.

Exemple :

`'days'.between(self.getCreationTimestamp(), self.getTraceabilityDateFormat(), '2024-01-01', 'yyyy-MM-dd')` retourne le nombre de jours séparant la date de création de l'objet `self` du 1er janvier 2024.

7.27.3. Déterminer la différence temporelle entre maintenant et la date de création ou de dernière modification d'un objet

Les services `getCreationTimeSince` et `getLastModificationTimeSince` permettent de calculer le temps entre la date de création ou la date de dernière modification d'un objet et le moment présent.

Ces deux services s'appliquent sur un objet du référentiel.

Ils prennent tous les deux en paramètre l'unité temporelle dans laquelle le résultat est souhaité. Les termes autorisés sont :

- years

- months
- weeks
- days
- hours
- minutes
- seconds

Ces services invalident le cache serveur d'une requête utilisant un de ces services.

```
Object.getCreationTimeSince(String unit) : String
```

Ce service calcule la quantité de temps qui s'est écoulée depuis la création d'un objet.

Exemple :

`self.getCreationTimeSince('days')` donne le nombre de jours qui se sont écoulés depuis la création de l'objet `self`.

```
Object.getLastModificationTimeSince(String unit) : String
```

Ce service calcule la quantité de temps qui s'est écoulée depuis la dernière modification d'un objet.

Exemple :

`self.getLastModificationTimeSince('months')` donne le nombre de mois qui se sont écoulés depuis la dernière modification de l'objet `self`.

7.27.4. Déterminer si une date précède ou suit une autre date

Les services `before` et `after` permettent de calculer si une date précède ou suit une autre date.

Ces services s'appliquent sur une date et renvoient un booléen.

```
String.before(String sourceFormat, String targetDate, String targetFormat) : Boolean
```

Appliqué sur une date ce service permet de préciser le format de la première date, la deuxième date ainsi que le format de la deuxième date.

```
String.before(String targetDate, String commonFormat) : Boolean
```

Appliqué sur une date ce service permet de préciser la deuxième date ainsi que le format commun aux deux dates.

```
String.before(String targetDate) : Boolean
```

Appliqué sur une date ce service permet de préciser uniquement la deuxième date. Le format de date configuré au niveau du serveur SmartEA est utilisé.

```
String.after(String sourceFormat, String targetDate, String targetFormat) : Boolean
```

Appliqué sur une date ce service permet de préciser le format de la première date, la deuxième date ainsi que le format de la deuxième date.

```
String.after(String targetDate, String commonFormat) : Boolean
```

Appliqué sur une date ce service permet de préciser la deuxième date ainsi que le format commun aux deux dates.

```
String.after(String targetDate) : Boolean
```

Appliqué sur une date ce service permet de préciser uniquement la deuxième date. Le format de date configuré au niveau du serveur SmartEA est utilisé.

Exemples :

```
objet1.getCreationTimestamp().before(objet2.getCreationTimestamp(),  
self.getTraceabilityDateFormat())
```

 retourne vrai si la date de création de l'objet 1 précède celle de l'objet 2.

```
objet1.getCreationTimestamp().after(self.getTraceabilityDateFormat(), '2024-01-01 00:00:00',  
'yyyy-MM-dd HH:mm:ss')
```

 retourne vrai si la date de création de l'objet 1 est postérieure au 1er Janvier 2024.

7.27.5. Déterminer si une propriété typée de type Date précède ou suit une autre

Les services `before` et `after` sur des propriétés typées permettent de déterminer si une propriété typée de type Date précède ou suit une autre.

```
TypedProperty.before(TypedProperty other) : Boolean
```

```
TypedProperty.after(TypedProperty other) : Boolean
```

Ces deux services s'appliquent sur une propriété typée et prennent en paramètre une seconde propriété typée. Les contraintes renseignées dans le prisme sont utilisées pour déterminer le format des dates.

```
AbstractTypedProperty.before(AbstractTypedProperty other, String dateFormat) : Boolean
```

```
AbstractTypedProperty.after(AbstractTypedProperty other, String dateFormat) : Boolean
```

Ces services peuvent également être utilisés sur des propriétés typées calculées. Dans ce cas le format de date doit être précisé.

```
AbstractTypedProperty.before(AbstractTypedProperty other, String dateFormat, String  
otherDateFormat) : Boolean
```

```
AbstractTypedProperty.after(AbstractTypedProperty other, String dateFormat, String  
otherDateFormat) : Boolean
```

Ces services sont une déclinaison des deux précédents pour lesquels il est possible de préciser le format de date des deux propriétés.

Exemples :

Supposant que deux objets `object1` et `object2` ont tous deux une propriété typée de type Date nommée `MyDateProperty` :

```
object1.getProperty('MyDateProperty').after(object2.getProperty('MyDateProperty'))
```

 retourne vrai si la date de `MyDateProperty` de l'objet1 est postérieure à la date de celle de l'objet2.

Si `object2` a une propriété typée calculée de type date `MyComputedDateProperty` dont on sait que le format est `<yyyy-MM-dd>` et que l'on veut savoir si la date dans la propriété `MyDateProperty` d'object1 est après la date de la propriété `MyComputedDateProperty` d'object2 alors la requête suivante permet de le savoir :

```
let object1Property = object1.getProperty('MyDateProperty') in let object2Property =  
object2.getProperty('MyComputedDateProperty') in object1Property.after(object2Property,  
object1Property.getConstraint(), 'yyyy-MM-dd')
```

Il est à noter que `object1Property.getConstraint()` doit retourner le format de date indiqué dans le prisme dans la description de la propriété typée `MyDateProperty`.

7.27.6. Addition et soustraction de temps sur une date

Les services `plus` et `moins` permettent d'ajouter ou soustraire un temps à une date.

Ces services s'appliquent sur une date et renvoient une date.

```
String.plus(Integer amount, String unit, String format) : String
```

```
String.minus(Integer amount, String unit, String format) : String
```

Le premier paramètre est le temps à ajouter ou soustraire à la date de départ. Le deuxième paramètre est l'unité temporelle du temps à ajouter ou soustraire (years, months, weeks, days, hours, minutes, seconds). Le troisième paramètre est le format de la date initiale et de la date résultat.

Exemple :

Si `self` a une date de création égale à 27/09/2023 13:37:20, alors `self.getCreationTimestamp().minus(30, 'days', self.getTraceabilityDateFormat())` retourne 28/08/2023 13:37:20.

7.27.7. Calculer le temps qui s'est écoulé depuis la date décrite dans une propriété typée Date

Le service `sinceNow` permet de calculer le temps écoulé depuis la date décrite dans une propriété typée de type Date.

Ce service s'applique sur une propriété typée et renvoie une date.

Ce service invalide le cache serveur d'une requête utilisant ce service.

```
TypedProperty.sinceNow(String unit) : String
```

Ce service s'applique sur une propriété typée non calculée.

```
AbstractTypedProperty.sinceNow(String unit, String dateFormat) : String
```

Ce service s'applique sur une propriété typée calculée ou non calculée.

Le premier paramètre est l'unité temporelle du temps (years, months, weeks, days, hours, minutes, seconds). Le deuxième paramètre du second service est le format de date attendue.

Exemple :

`self.getProperty('MyDateProperty').sinceNow('days')` retourne le nombre de jours qui se sont écoulés depuis la date décrite dans la propriété `MyDateProperty`.

7.28. Evaluer une requête AQL

Le service `eval` permet d'évaluer une expression AQL sur un objet sémantique.

```
Object.eval(String query) : Object
```

Exemple :

```
aql:self.eval(self.getPropertyValue('AQL query'))
```

L'exemple précédent évalue une requête AQL stockée dans la propriété nommée «AQL query» de l'objet `self`.

7.29. Récupérer l'icone d'un élément sémantique

Attention : les services décrits dans cette section sont disponibles uniquement coté serveur.

Le service `getIconWebURL` permet de récupérer l'url de l'icone d'un objet sémantique.

```
Object.getIconWebURL() : String
```

Exemple :

```
self.getIconWebURL()
```

La requête précédente renvoie l'url de l'icone de l'objet self.

Le service `makeIconHTML` appliqué sur un objet sémantique renvoie le code HTML permettant d'afficher l'icone d'un objet sémantique (par exemple pour afficher une icone dans l'artefact *Smart Request* ou le widget *List*).

```
Object.makeIconHTML() ; String
```

7.30. Récupérer une image de la bibliothèque d'images

Attention : les services décrits dans cette section sont disponibles uniquement coté serveur.

Le service `getImageWebURL` appliqué sur le nom (et chemin) d'une image de la bibliothèque d'images permet de récupérer l'url de l'image.

```
String.getImageWebURL() : String
```

Exemple :

```
'Folder#1/image01.png'.getImageWebURL()
```

La requête précédente renvoie l'url de l'image nommée `image01.png` qui se trouve dans le répertoire `Folder#1`.

Le service `makeImageHTML` appliqué sur le nom (et chemin) d'une image de la bibliothèque d'images renvoie le code HTML permettant d'afficher l'image (par exemple dans l'artefact *Smart Request* ou le widget *List*).

```
String.makeImageHTML(int width, int height) : String
```

Les paramètres **width** et **height** permettent de préciser la largeur et la hauteur de l'image.

8. Services du modules ArchiMate

Cette section présente les services apportés par le module ArchiMate.

8.1. Relations ArchiMate

Pour faciliter l'écriture de requêtes, le métamodèle ArchiMate fournit deux services pour chaque type de relation :

- l'un permettant de récupérer les éléments reliés via une relation,
- l'autre fournissant la relation inverse.

Ces services sont :

- Relation de Spécialisation
 - specializationOf()
 - specializes()
- Relation de Composition
 - partOf()
 - composedOf()
- Relation d'Agrégation
 - aggregatedBy()
 - aggregates()
- Relation de Déclenchement
 - triggeredBy()
 - triggers()
- Relation de Flux
 - flowsFrom()
 - flowsTo()
- Relation d'Accès
 - accessedBy()
 - accesses()
- Relation d'Association
 - associatedWith() (retourne une fusion des elements entrants et sortants)
- Relation d'Affectation
 - assignedFrom()
 - assignedTo()
- Relation de Réalisation
 - realizedBy()
 - realizes()
- Relation de Service
 - servedBy()
 - serves()
- Relation d'Influence
 - influencedBy()
 - influences()

Tous ces services acceptent une chaîne de caractères permettant de préciser le stéréotype que la relation doit utiliser.

```
associatedWith('nom d'un stéréotype')
```

On peut également ajouter à ces services deux paramètres qui indiquent le nom d'un attribut sur lequel filtrer les relations ainsi qu'une expression régulière filtrant la valeur de ces attributs.

```
associatedWith('nom d'un stéréotype', 'name', '^Prefixe de relation.*$')
```

Si on ne veut pas préciser de stéréotype on peut indiquer null à la place d'une chaîne de caractères.

```
associatedWith(null, 'name', '^Prefixe de relation.*$')
```

Le nom d'attribut peut être la clef d'une propriété dynamique (typée ou pas) ainsi qu'un attribut permanent de l'objet tel que name, id et autres. Les attributs considérés sont ceux portés par les relations et non leur source/cible.

8.2. Retrouver des éléments ArchiMate par leur nom

Appliqué sur un objet quelconque, le service childrenByName renvoie tous les éléments ArchiMate enfants (liés par une relation de contenance) de cet objet et ayant pour nom le nom donné en paramètre. Le nom peut être une expression régulière.

```
Object.childrenByName(String name) : Set<ArchiMateComponent>
```

Exemple :

Appliqué sur un répertoire, `aql:self.childrenByName('CRM')` renvoie tous les éléments ArchiMate du répertoire et de ses sous répertoires contenant la chaîne de caractères **CRM** dans leur nom.

Le service elementsByName filtre parmi une collection d'éléments ArchiMate, l'ensemble des éléments ayant le nom spécifié en paramètre. Le nom peut être une expression régulière.

```
Collection<ArchiMateComponent>->elementsByName(String name) : Set<ArchiMateComponent>
```

Exemple :

`aql:self.childrenByType('ApplicationComponent')->elementsByName('CRM')` filtre la liste obtenue grâce au service **childrenByType('ApplicationComponent')** en ne retenant que les éléments contenant la chaîne de caractères **CRM** dans leur nom.

8.3. Retrouver un élément ArchiMate d'un type donné par son nom

Appliqué sur un objet quelconque, le service xyzByName renvoie le premier élément ArchiMate de type xyz lié par une relation de contenance à cet objet et ayant pour nom le nom donné en paramètre. Le nom peut être une expression régulière.

```
EObject.xyzByName(String name) : XYZ
```

Exemple :

Appliqué sur un répertoire, `aql:self.applicationComponentByName('CRM')` renvoie le premier élément ArchiMate de type ApplicationComponent et de nom **CRM** si celui-ci existe dans le répertoire ou un de ses sous répertoires.

Les services disponibles sont les suivants :

```
EObject.applicationCollaborationByName(String name) : ApplicationCollaboration
```

```
EObject.applicationComponentByName(String name) : ApplicationComponent
```

```
EObject.applicationEventByName(String name) : ApplicationEvent
```

```
EObject.applicationFunctionByName(String name) : ApplicationFunction
```

```
EObject.applicationInteractionByName(String name) : ApplicationInteraction
```

```
EObject.applicationInterfaceByName(String name) : ApplicationInterface
```

```
EObject.applicationProcessByName(String name) : ApplicationProcess
```

```
EObject.applicationServiceByName(String name) : ApplicationService
```

EObject.artifactByName(String name) : Artifact

EObject.assessmentByName(String name) : Assessment

EObject.businessActorByName(String name) : BusinessActor

EObject.businessCollaborationByName(String name) : BusinessCollaboration

EObject.businessEventByName(String name) : BusinessEvent

EObject.businessFunctionByName(String name) : BusinessFunction

EObject.businessInteractionByName(String name) : BusinessInteraction

EObject.businessInterfaceByName(String name) : BusinessInterface

EObject.businessObjectByName(String name) : BusinessObject

EObject.businessProcessByName(String name) : BusinessProcess

EObject.businessRoleByName(String name) : BusinessRole

EObject.businessServiceByName(String name) : BusinessService

EObject.capabilityByName(String name) : Capability

EObject.communicationNetworkByName(String name) : CommunicationNetwork

EObject.constraintByName(String name) : Constraint

EObject.contractByName(String name) : Contract

EObject.courseOfActionByName(String name) : CourseOfAction

EObject.dataObjectByName(String name) : DataObject

EObject.deliverableByName(String name) : Deliverable

EObject.deviceByName(String name) : Device

EObject.distributionNetworkByName(String name) : DistributionNetwork

EObject.driverByName(String name) : Driver

EObject.equipmentByName(String name) : Equipment

EObject.facilityByName(String name) : Facility

EObject.gapByName(String name) : Gap

EObject.goalByName(String name) : Goal

EObject.implementationEventByName(String name) : ImplementationEvent

EObject.materialByName(String name) : Material

EObject.meaningByName(String name) : Meaning

EObject.nodeByName(String name) : Node

EObject.outcomeByName(String name) : Outcome

```
EObject.pathByName(String name) : Path
EObject.plateauByName(String name) : Plateau
EObject.principleByName(String name) : Principle
EObject.productByName(String name) : Product
EObject.representationByName(String name) : Representation
EObject.requirementByName(String name) : Requirement
EObject.resourceByName(String name) : Resource
EObject.stakeholderByName(String name) : Stakeholder
EObject.systemSoftwareByName(String name) : SystemSoftware
EObject.technologyCollaborationByName(String name) : TechnologyCollaboration
EObject.technologyEventByName(String name) : TechnologyEvent
EObject.technologyFunctionByName(String name) : TechnologyFunction
EObject.technologyInteractionByName(String name) : TechnologyInteraction
EObject.technologyInterfaceByName(String name) : TechnologyInterface
EObject.technologyProcessByName(String name) : TechnologyProcess
EObject.technologyServiceByName(String name) : TechnologyService
EObject.valueByName(String name) : Value
EObject.valueStreamByName(String name) : ValueStream
EObject.workpackageByName(String name) : WorkPackage
```

8.4. Rechercher les éléments ArchiMate atteignables depuis un élément ArchiMate

Le service `reachableElements` renvoie tous les éléments ArchiMate qui sont atteignables depuis l'élément de contexte et qui sont :

- du (des) type(s) donné(s) en paramètre (expression régulière),
- atteignables en suivant les types de relation donnés en paramètre (expression régulière).

```
ArchiMateComponent.reachableElements(String classes, String relations) :  
Set<ArchiMateComponent>
```

Exemples :

`aql:self.reachableElements('ApplicationComponent','Serving|Specialization')` renvoie les composants applicatifs atteignables par des relations de servitude ou de spécialisation.

`aql:self.reachableElements('ApplicationComponent','')` renvoie les composants applicatifs atteignables depuis l'objet de contexte (quelque soit le type de relation).

`aql:self.reachableElements('', 'Serving|Specialization')` renvoie les objets atteignables depuis l'objet de contexte par des relations de servitude ou de spécialisation.

8.5. Rechercher des chemins

Le service `pathesTo` recherche tous les chemins depuis l'élément de contexte vers tout élément dont le type correspond à l'un des types spécifiés en paramètre (expression régulière).

```
ArchimateComponent.pathesTo(String classes) : Set<ArchimateComponent>
```

Exemple :

Appliqué sur un élément métier, `aql:self.pathesTo('SystemSoftware|Device')` retourne tous les éléments ArchiMate faisant partie de chemins partant de l'élément métier et arrivant sur un **Logiciel** ou une **Machine**.

Le service `pathesTo` suivant recherche tous les chemins depuis l'élément de contexte vers tout élément dont :

- le type correspond à l'un des types spécifiés et
- en traversant uniquement les relations spécifiées.

Les types et les relations peuvent être spécifiés sous forme d'expression régulière.

```
ArchimateComponent.pathesTo(String classes, String relations) : Set<ArchimateComponent>
```

Exemple :

Appliqué sur un élément métier, `aql:self.pathesTo('SystemSoftware|Device', 'Serving|Specialization')` retourne tous les éléments ArchiMate faisant partie de chemins partant de l'élément métier et arrivant sur un **Logiciel** ou une **Machine** en suivant les relations de servitude ou de spécialisation.

Le service `pathesToLayer` recherche tous les chemins depuis l'élément de contexte vers les éléments appartenant à la couche d'architecture spécifié en paramètre.

La couche d'architecture peut prendre une des valeurs suivantes :

- STRATEGY
- MOTIVATION
- BUSINESS
- APPLICATION
- TECHNOLOGY
- PHYSICAL
- IMPLEMENTATION
- COMPOSITE

```
ArchimateComponent.pathesToLayer(String layer) : Set<ArchimateComponent>
```

Exemple :

Appliqué sur un élément métier, `aql:self.pathesToLayer('APPLICATION')` retourne tous les éléments ArchiMate faisant partie de chemins partant de l'élément métier et arrivant sur un élément applicatif.

9. Documentation officielle AQL

Vous trouverez à l'URL suivante l'ensemble des [services fournis par AQL](https://www.eclipse.org/acceleio/documentation/aql.html) avec des exemples explicatifs : <https://www.eclipse.org/acceleio/documentation/aql.html>

AQL est construit au dessus de

- Eclipse Acceleio qui fournit un ensemble de services génériques de bas niveau pour naviguer dans un modèle conforme à un métamodèle
- ainsi que des services apportés par [OCL](https://www.omg.org/spec/OCL/) (<https://www.omg.org/spec/OCL/>).

Le guide des [services génériques de bas niveau apportés par Acceleio](https://wiki.eclipse.org/Acceleio/Acceleio_Operations_Reference) est disponible à cette URL : https://wiki.eclipse.org/Acceleio/Acceleio_Operations_Reference

La liste des [services apportés par OCL](https://wiki.eclipse.org/Acceleio/OCL_Operations_Reference) est disponible ici : https://wiki.eclipse.org/Acceleio/OCL_Operations_Reference